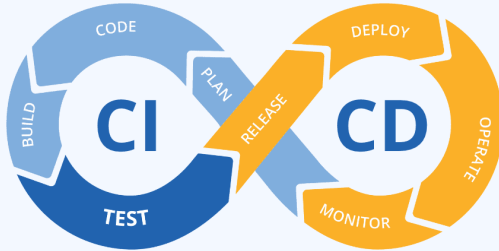
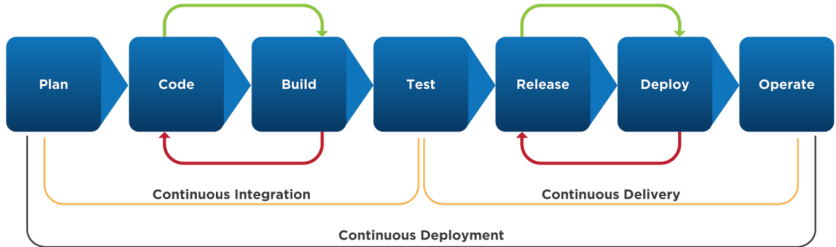


Razvoj softvera. Git. Testovi. *Code coverage*. Integracija na *pipeline*.

Katedra za automatsko upravljanje
Fakultet tehničkih nauka
Novi Sad

4. maj 2026.





Planning

- Requirement finalization
- Updates & new changes
- Architecture & design
- Task assignment
- Timeline finalization

Code

- Development
- Configuration finalization
- Check-in source code
- Static-code analysis
- Automated review & peer review

Build

- Compile code
- Unit testing
- Code-metrics
- Build container images or package
- Preparation or update in deployment templated
- Create or update monitor dashboards

Test

- Integration test with other component
- Load & stress test
- UI testing
- Penetration testing
- Requirement testing

Release

- Preparing release notes
- Version tagging
- Code freeze
- Feature freeze

Deploy

- Updating the infrastructure i.e staging, production
- Verification on deployment i.e smoke tests

Operate

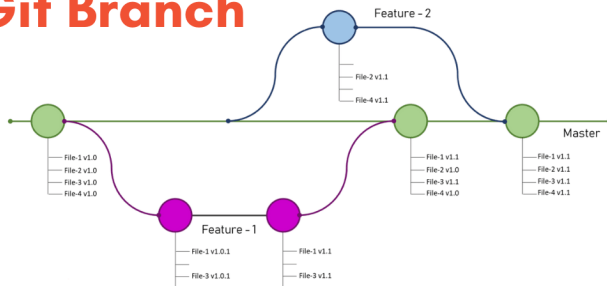
- Monitor designed dashboard
- Alarm triggers
- Automatic critical events handler
- Monitor error logs

- prilikom razvoja bilo kog softvera, na njemu radi više razvojnih programera i testera na različitim računarima, *Git* omogućava sinhronizaciju i istovremeni rad
- ukoliko programeri nisu menjali iste funkcionalnosti samo će se dodati nove izmene a ukoliko jesu onda će morati da se reše konflikti do kojih je došlo
- sprečava gubljenje podataka



¹Instalacija: <https://git-scm.com/install/windows>

How To Create a Git Branch



Inicijalizacija:

- ***git init***
- ***git clone <repository_url>***

Najčešće ručno napravimo repo pa ga kloniramo.

HEAD -> gde sam sada

origin -> skraćeno ime za udaljeni repozitorijum

Grane:

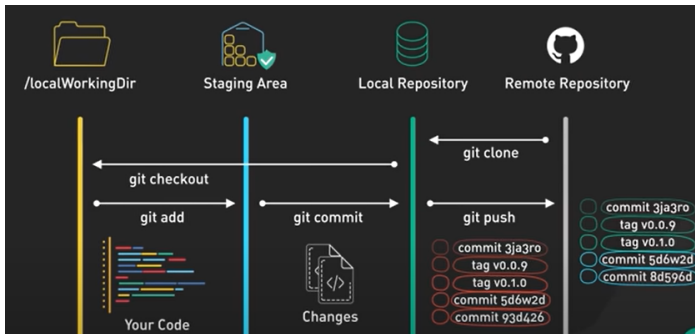
- ***git branch feature/...*** -> kreiranje grane
- ***git branch*** -> sve grane i na kojoj sam sada
- ***git checkout feature/...*** -> menjanje grane
- ***git branch -d bugfix/...*** -> brisanje grane

git log -> lista prethodnih *commit*-ova i gde se nalazim u odnosu na njih

git --help -> lista osnovnih komandi

Repozitorijumi

Pravljenjem grana možemo raditi na više stvari istovremeno, pokretanjem ***git add <files>*** i ***git commit -m "Commit description"*** lokalno dobijamo te izmene. Za njihovo propagiranje na remote repozitorijum koristimo ***git push origin feature/....***



Repozitorijumi

Informacije o tome koje smo fajlove menjali a da nisu *staged*-ovani, koje smo dodali i spremni su za *commit*, u kojim fajlovima se nalaze konflikti itd:

git status

Ako smo dodali fajlove koje nismo želeli, opozivanje se vrši komandom:

git restore --staged <files>.

Razlike radnih i *staged* fajlova:

git diff

Razlike prethodna dva *commit*-a:

git diff HEAD HEAD~1

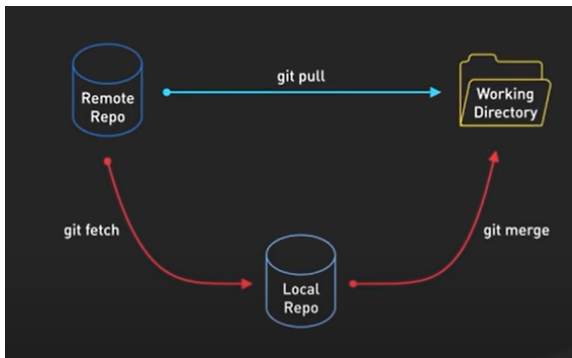
Od *N* commit-ova napraviti jedan:

git rebase -i HEAD N, squash-ujemo

Sve fajlove koje ne želimo da pošaljemo na repozitorijum treba da stavimo u **.gitignore** i njihove promene neće biti praćene.

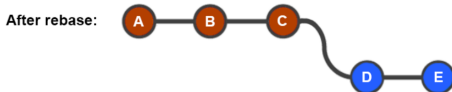
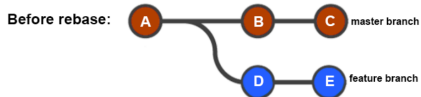
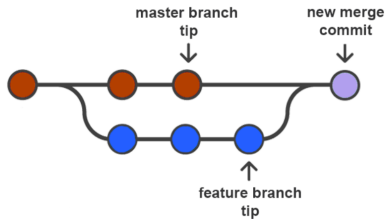
Poravnavanje sa udaljenim repozitorijumom

Za povlačenje izmena sa udaljenog repozitorijuma koristimo **git pull**.



Ukoliko dođe do konflikta potrebno ih je razrešiti da bi *pull* bio uspešan.

Merge vs rebase

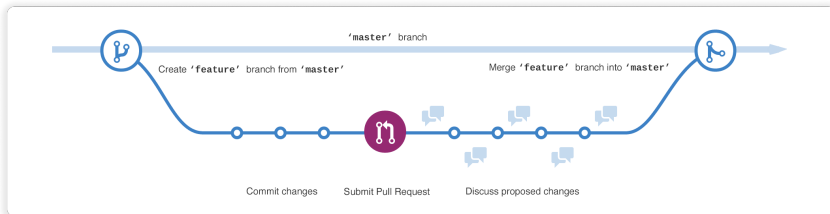


Undo opcije

- **git reset** -> vraćamo se unazad N commit-a
- **git stash**, ..., **git stash pop** -> sačuvamo izmene pa ih sa stash pop vratimo lokalno
- **git cherry-pick <hash>** -> dodavanje određenog *commit*-a na vrh naše grane, prvo napravimo *commit* pa resetujemo ili menjamo nešto pa se uvek možemo vratiti u sačuvano stanje kopiranjem njegovog heša koji ćemo naći preko *git log* komande
- **git revert** -> pravi novi *commit* koji poništava prethodni
- **git commit --amend** -> ne pravimo novi *commit*, izmene pripajamo poslednjem
- **git rm --cached <files>** pa **git commit --amend** -> uklanja *commit*-ovane fajlove
- **git restore** -> odbacuje nekomitovane lokalne promene u radnom direktorijumu
- **git rebase** -> menja istoriju *commit*-ova

Pull request / Merge request

Da bi se načinjene izmene primenile na udaljenom repozitorijumu potrebno je da napravimo *PR*. Kada se sve diskusije zatvore, izmene će biti primenjene.



Git flow - verzionisanje

Recimo da se trenutno proizvod nalazi na 1.4.0 verziji.

- nakon dodavanja nove funkcionalnosti (spajanja *feature* grane sa *main/master/develop* granom) kreira se *alpha* verzija, npr. 1.5.0~alpha.172
- kad treba da izađe nova verzija proizvoda pravi se grana za 1.5.0 verziju
- nakon pravljenja *release* grane, dodavanje određenih funkcionalnosti će rezultirati *beta* verzijom, npr. 1.5.0~beta.1
- pred sam *release* pravi se *tag* i *release notes* svih *CR*-ova
- ako se otkriju određeni *bug*-ovi od strane korisnika, na *support* grani se njihovim rešavanjem se dobijaju *hotfix* verzije, npr. 1.5.1
- podrška za određene verzije traje onoliko koliko odluči vlasnička struktura tog proizvoda (npr. kažu da su verzije manje od 1.3.0 *out of support*)
- kada dođe do veće promene u proizvodu (promena SDK-a) menja se prvi broj, npr. 2.0.0 može doći nakon 1.750.0

Svaka implementirana funkcionalnost se mora validirati pre njenog dodavanja (i pre zatraživanja review-a na MR-u). Testovi se dele na funkcionalne (da li program radi ono što treba) i vanfunkcionalne/nefunkcionalne (kako program radi).

Tipovi funkcionalnih testova:

- Unit - jedna metoda, proveravamo da li je izlaz očekivan
- Integration - više komponenti zajedno
- System - test čitavog sistema
- Acceptation - test da li su zahtevi zadovoljeni (radi klijent ili PO)

Tipovi vanfunkcionalnih testova:

- Performance - brzina i opterećenja
- Usability - da li je aplikacija laka za korišćenje
- Compatibility - testiranje na različitim uređajima i sistemima
- Security - provera bezbednosti

- Testiranje najmanjih jedinica koda (metode, klase) u izolaciji, bez spoljašnjih zavisnosti
- Cilj: provera ispravnosti logike
- Omogućavaju lakše refaktorisanje i održavanje koda
- Pokreću se automatski (CI/CD) kada god se *push*-uje kod na repozitorijum (integriše se na GitLab pipeline ili se doda GitHub Action)
- Svaki test prati **AAA obrazac**:
 - Arrange – priprema ulaza i objekata
 - Act – poziv metode koja se testira
 - Assert – provera rezultata

- Podržavaju:
 - Attribute ([Fact], [Test], [Theory])
 - Assert metode (provera rezultata)
 - setup/teardown mehanizmi
 - Grupisanje i organizaciju testova
 - Assert metode:
 - Assert.Equal(expected, actual)
 - Assert.True(condition)
 - Assert.Throws<Exception>(…)
 - Testovi treba da budu:
 - deterministički (isti rezultat svaki put)
 - nezavisni (bez zavisnosti između testova)
 - brzi
- Framework: xUnit, NUnit - stariji, MSTest - Microsoft-ov zvanični framework

- Mock objekti su simulirane zavisnosti koje zamenjuju realne komponente, koje smo već testirali pa nema potrebe da se izvršava čitav kod opet, treba nam samo potvrda se pozvala određena metoda prilikom verifikacije trenutnog *unit*-a
- Koriste se za izolaciju sistema koji se testira (npr. baza, API, servisi)
- Omogućavaju:
 - kontrolu ponašanja (npr. vraćanje vrednosti)
 - simulaciju grešaka (exception) ili specifičnih *edge case*-ova (npr. timeout prilikom pristupa servisu)
 - verifikaciju interakcije (da li je metoda pozvana)
- Popularne biblioteke: Moq, NSubstitute, FakeItEasy

- **Behavior control:**

- `Setup(x => x.Method()).Returns(value)`
- `Setup(x => x.Method()).Throws<Exception>()`

- **Argument matchers:**

- `It.IsAny<T>()`
- `It.Is<T>(x => x > 5)`

- **Verifikacija poziva:**

- `mock.Verify(x => x.Method(), Times.Once)`

- **Sequence testing:**

- vraćanje različitih vrednosti po pozivu

- **Najbolja praksa:**

- mock-ovati interfejsse
- koristiti virtual metode ako je klasa

Code coverage

- Mera koliko je koda izvršeno tokom testiranja
- Ne meri kvalitet testova već obim pokrivenosti
- Koristi se za identifikaciju netestiranih delova
- Najbitniji Branch i Function Coverage
- U sistemima se najčešće teži pokrivenosti koda 75-80%, u baš velikim sistemima 65% može biti prihvatljivo
- Pokrivenost od 100% znači da smo verovatno previše vremena proveli pišući testove, nema smisla sve testirati

Tipovi Code Coverage

- **Function coverage** – procenat pozvanih funkcija/metoda
- **Statement coverage** – procenat izvršenih linija koda
- **Line coverage** – da li je linija izvršena
- **Branch coverage** – pokrivenost svih grana (if/else)
- **Condition coverage** – svaka logička komponenta true/false
- **Path coverage** – sve moguće putanje izvršavanja
- **MC/DC** – svaka logička promenljiva utiče na odluku
- **Loop coverage** – 0, 1 i više iteracija petlje
- **Patch coverage** – pokrivenost novog koda (commit)

Kako podesiti sve

```
# Napraviti projekte
mkdir StudentApp
cd StudentApp
dotnet new sln
dotnet new classlib -n StudentApp
dotnet sln add StudentApp
dotnet new xunit -n StudentApp.Tests
dotnet sln add StudentApp.Tests

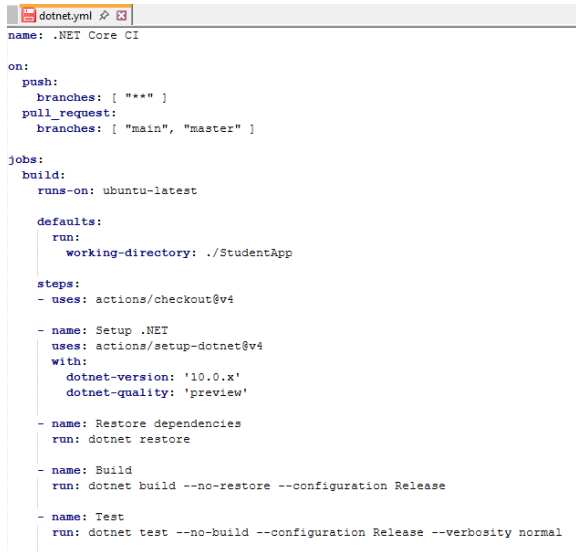
# Povezati testove sa aplikacijom i instalirati potrebne pakete
dotnet add StudentApp.Tests/StudentApp.Tests.csproj reference StudentApp/StudentApp.csproj
cd StudentApp.Tests
dotnet add package Moq
dotnet add package coverlet.collector
dotnet tool install --global dotnet-reportgenerator-globaltool
dotnet add package coverlet.msbuild

# Pokretanje programa i generisanje code coverage report-a
dotnet test --collect:"XPlat Code Coverage"

reportgenerator -reports:"**/coverage.cobertura.xml" -targetdir:"coveragereport" -reporttypes:Html
```

- Platforma za **CI/CD**
- Omogućava automatsko pokretanje procesa na događaje:
 - push, pull request, merge, schedule
- Tipični use-case:
 - build projekta
 - pokretanje unit testova
 - merenje code coverage
 - deployment aplikacije
- Prednosti:
 - rano otkrivanje grešaka
 - automatizacija procesa
 - standardizacija build/test pipeline-a
- Workflow se definiše u: **.github/workflows/*.yaml**
- Moguće je dodatno podesiti repozitorijum dodavanjem *protection* pravila, npr. onemogućiti spajanje sa glavnom granom ukoliko su testovi pali, postaviti da se testovi smatraju neuspehim ukoliko je code coverage ispod 80% itd.

Primer: automatsko pokretanje build-a i testova na push



```
dotnet.yml
name: .NET Core CI

on:
  push:
    branches: [ "*" ]
  pull_request:
    branches: [ "main", "master" ]

jobs:
  build:
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: ./StudentApp

    steps:
      - uses: actions/checkout@v4

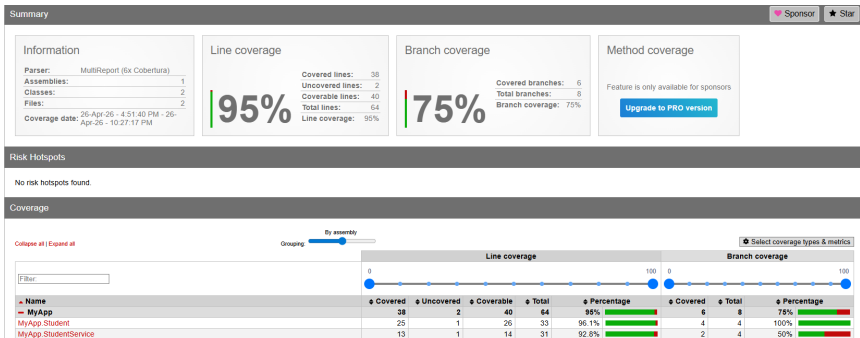
      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '10.0.x'
          dotnet-quality: 'preview'

      - name: Restore dependencies
        run: dotnet restore

      - name: Build
        run: dotnet build --no-restore --configuration Release

      - name: Test
        run: dotnet test --no-build --configuration Release --verbosity normal
```

Slika: Primer .yaml fajla



Slika: Pregled code coverage-a korišćenjem Report Generatora

#	Line	Line coverage
	1	using System;
	2	using System.Collections.Generic;
	3	using System.Text;
	4	
	5	namespace MyApp
	6	{
	7	public class StudentService
	8	{
	9	private readonly IStudentRepository studentRepository;
1	10	public StudentService(IStudentRepository studentRepository)
1	11	{
1	12	this.studentRepository = studentRepository;
1	13	}
	14	public bool PrepareStudentForExam(int studentId, int hours)
1	15	{
1	16	var student = studentRepository.GetStudentById(studentId);
	17	
1	18	if (student == null) return false;
	19	
1	20	student.Learn(hours);
	21	
1	22	if (student.IsTestPassed())
1	23	{
1	24	studentRepository.Save(student);
1	25	return true;
	26	}
	27	
0	28	return false;
1	29	}
	30	}
	31	}

Slika: Pregled code coverage-a korišćenjem Report Generatora

```
Test
1 ▶ Run dotnet test --no-build --configuration Release \
11 [coverlet] _mapping file name: 'CoverletSourceRootsMapping_MyApp.Tests'
12 Test run for /home/runner/work/vezbe4/vezbe4/ConsoleApp1/StudentApp/MyApp.Tests/bin/Release/net10.0/MyApp.Tests.dll (.NETCoreApp,Version=v10.0)
13 A total of 1 test files matched the specified pattern.
14
15 Passed! - Failed: 0, Passed: 5, Skipped: 0, Total: 5, Duration: 81 ms - MyApp.Tests.dll (net10.0)
16 [coverlet]
17 Calculating coverage result...
18 Generating report '/home/runner/work/vezbe4/vezbe4/ConsoleApp1/StudentApp/MyApp.Tests/coverage.cobertura.xml'
19
20 +-----+-----+-----+-----+
21 | Module | Line | Branch | Method |
22 +-----+-----+-----+-----+
23 | MyApp | 91.3% | 66.66% | 90% |
24 +-----+-----+-----+-----+
25
26 +-----+-----+-----+-----+
27 |      | Line | Branch | Method |
28 +-----+-----+-----+-----+
29 | Total | 91.3% | 66.66% | 90% |
30 +-----+-----+-----+-----+
31 | Average | 91.3% | 66.66% | 90% |
32 +-----+-----+-----+-----+
```

Slika: Izgled pipeline-a koji se pokrene nakon svakog push-a

Scrum

Najčešći oblik unutrašnje organizacije u firmama za rukovođenje projektima.

- dnevni sastanci (šta je rađeno juče i šta će se raditi danas)
- 3-nedeljni sastanci, **sprint** review and planning
- 6-mesečni sastanci, **release** review and planning
- postoji i *backlog* za neprioritetne *task*-ove

Release ima devet *sprint*-ova, sedmi se naziva *feature freeze* i nakon njega nema dodavanja novih funkcionalnosti: testiraju se postojeće, refaktoriše se kod, pravi se dokumentacija i planovi za naredni release (*analyze task*-ovi).



Slika: Pozicije u timu po *scrum*-u